

Lab 7: Advanced Mocking with Moq Callbacks

Objective

Learn how to use advanced mocking techniques, such as callbacks, to test classes that involve complex interactions.

Prerequisites

- Completion of **Lab 6** or familiarity with testing asynchronous code.
- Familiarity with mocking frameworks like Moq.

Instructions

NOTE: You're going to be modifying the OrderProcessor code in this lab. I'm having you add a new order method called **ProcessOrderAndShip()** and having you leave the existing **ProcessOrder()** method alone and untouched. You probably wouldn't do this in real life. You'd probably combine these two methods into a single method...

...but I didn't want you to have to do a TON of extra typing and modifying the existing tests for OrderProcessor. So for the most part, the changes in this lab are adding code instead of modifying existing code.

(You're welcome. 😊)

Step 1: Create an IShippingService and OrderProcessor Classes

1. In the `NunitLab` project, update or create an interface `IShippingService.cs`:

```
namespace NunitLab.Api;

public interface IShippingService
{
    decimal CalculateShippingCost(string destination, double weight);
    bool ValidateShippingDetails(string destination, double weight);
}
```

2. Modify your existing `OrderProcessor` class code to use the `IShippingService`:

```
namespace NunitLab.Api;

public class OrderProcessor
{
    private readonly IEmailService _emailService;
    private readonly IShippingService _shippingService;
```

```

public OrderProcessor(
    IShippingService shippingService,
    IEmailService emailService)
{
    _emailService = emailService;
    _shippingService = shippingService;
}

public bool ProcessOrder(string orderId, string customerEmail)
{
    // Logic to process the order...
    if (string.IsNullOrEmpty(orderId)) return false;

    // Send confirmation email
    _emailService.SendEmail(
        customerEmail,
        "Order Confirmation",
        $"Your order {orderId} has been processed.");

    return true;
}

public decimal ProcessOrderAndShip(
    string orderId, string customerEmail,
    string destination, double weight)
{
    if (_shippingService.ValidateShippingDetails(
        destination, weight) == false)
    {
        throw new InvalidOperationException(
            "Invalid shipping details.");
    }

    var shippingCost =
        _shippingService.CalculateShippingCost(
            destination, weight);

    _emailService.SendEmail(
        customerEmail,
        "Order Confirmation",
        $"Your order will be shipped to {destination} with a shipping cost of {shippingCost:C}.");

    return shippingCost;
}
}

```

Step 2: Write Tests with Callback Mocking

1. In the `NunitLab.UnitTests` project, update the `OrderProcessorTests.cs` file:

```
using Moq;
using NunitLab.Api;

namespace NunitLab.UnitTests;

[TestFixture]
public class OrderProcessorTests
{
    private OrderProcessor? _systemUnderTest;
    private Mock<IEmailService>? _emailServiceMock;
    private Mock<IShippingService>? _shippingServiceMock;

    public OrderProcessor SystemUnderTest
    {
        get
        {
            if (_systemUnderTest == null)
            {
                _systemUnderTest = new OrderProcessor(
                    ShippingServiceMock.Object,
                    EmailServiceMock.Object);
            }

            Assert.That(_systemUnderTest, Is.Not.Null);
            Assert.That(_emailServiceMock, Is.Not.Null);

            return _systemUnderTest;
        }
    }

    public Mock<IEmailService> EmailServiceMock
    {
        get
        {
            if (_emailServiceMock == null)
            {
                _emailServiceMock = new Mock<IEmailService>();
            }

            Assert.That(_emailServiceMock, Is.Not.Null);

            return _emailServiceMock;
        }
    }
}
```

```

public Mock<IShippingService> ShippingServiceMock
{
    get
    {
        if (_shippingServiceMock == null)
        {
            _shippingServiceMock = new Mock<IShippingService>();
        }

        Assert.That(_shippingServiceMock, Is.Not.Null);

        return _shippingServiceMock;
    }
}

[SetUp]
public void SetUp()
{
    _systemUnderTest = null;
    _emailServiceMock = null;
    _shippingServiceMock = null;
}

[Test]
public void ProcessOrder_WhenOrderIdIsValid_ShouldSendEmail()
{
    // Arrange
    string orderId = "12345";
    string customerEmail = "customer@example.com";

    // Act
    var actual = SystemUnderTest.ProcessOrder(orderId, customerEmail);

    // Assert
    Assert.That(actual, Is.True);

    EmailServiceMock.Verify(
        x => x.SendEmail(
            customerEmail,
            "Order Confirmation",
            $"Your order {orderId} has been processed."),
        Times.Once);
}

[Test]
public void ProcessOrder_WhenOrderIdIsInvalid_ShouldNotSendEmail()
{
    // Arrange
    string orderId = "";

```

```

        string customerEmail = "customer@example.com";

        // Act
        var actual = SystemUnderTest.ProcessOrder(orderId, customerEmail);

        // Assert
        Assert.That(actual, Is.False);
        EmailServiceMock.Verify(x => x.SendEmail(It.IsAny<string>(), It.IsAny<string>()),
            It.IsAny<string>(), Times.Never);
    }

    [Test]
    public void
ProcessOrderAndShip_ShouldCallCalculateShippingCost_WithCorrectParameters()
    {
        // Arrange
        var orderId = "12345";
        var email = "customer@email.com";
        var destination = "Decimal Point, RI";
        var weight = 5.0;
        var expectedShippingCost = 10.0m;

        ShippingServiceMock.Setup(s => s.CalculateShippingCost(destination, weight))
            .Returns(expectedShippingCost);
        ShippingServiceMock.Setup(s => s.ValidateShippingDetails(destination, weight))
            .Returns(true);

        // Act
        var actualShippingCost = SystemUnderTest.ProcessOrderAndShip(orderId, email,
            destination, weight);

        // Assert
        Assert.That(actualShippingCost, Is.EqualTo(expectedShippingCost));
        ShippingServiceMock.Verify(s => s.CalculateShippingCost(destination, weight),
            Times.Once);
    }

    [Test]
    public void ProcessOrderAndShip_ShouldSendEmail_WithShippingDetails()
    {
        // Arrange
        var orderId = "34562";
        var email = "customer@email.com";
        var destination = "Backflip, MT";
        double weight = 2.5;
        decimal shippingCost = 15.0m;

        ShippingServiceMock.Setup(s => s.CalculateShippingCost(destination, weight))
            .Returns(shippingCost);
    }

```

```

ShippingServiceMock.Setup(s => s.ValidateShippingDetails(destination, weight))
    .Returns(true);

EmailServiceMock.Setup(e => e.SendEmail(email, It.IsAny<string>(),
It.IsAny<string>()))
    .Callback<string, string, string>((recipient, subject, body) =>
    {
        Assert.That(subject, Is.EqualTo("Order Confirmation"));
        Assert.That(body, Contains.Substring($"shipping cost of
{shippingCost:C}"));
        Assert.That(email, Is.EqualTo(recipient));
    });

// Act
_ = SystemUnderTest.ProcessOrderAndShip(orderId, email, destination, weight);

// Assert
EmailServiceMock.Verify(e => e.SendEmail(email, It.IsAny<string>(),
It.IsAny<string>()), Times.Once);
}
}

```

Step 3: Run the Tests

1. Open the **Test Explorer** in Visual Studio.
2. Run all tests and verify that:
 - `CalculateShippingCost` is called with the correct parameters.
 - Email content is validated through a callback.
 - Validation logic for the order details is mocked and pushed off to the `IShippingService` in order to keep our tests focused

Outcome

Students will:

- Understand how to use callbacks in mocking frameworks to validate dynamic interactions.
- Learn to test classes with complex dependencies and interactions.